The Journal of Open Source Software

individual: An R package for individual based epidemiological models

Giovanni D. Charles[*1] **and Sean L. Wu**[2]

1 MRC Centre for Global Infectious Disease Analysis, Abdul Latif Jameel Institute for Disease and Emergency Analytics (J-IDEA), Imperial College London, London, UK. **2** Division of Epidemiology and Biostatistics, School of Public Health, University of California, Berkeley, CA 94720, USA

DOI: 10.21105/joss.03377

Software

- Review
- Repository
- Archive

Editor: Pending Editor

Submitted: 16 June 2021
Published: 16 June 2021

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Complex stochastic models are a crucial tool for many tasks in public health, and especially in infectious disease epidemiology (Ganyani et al., 2021). Such models can formalize theory, generate synthetic data, evaluate counterfactual scenarios, forecast trends, and be used for statistical inference. Individual-based models (IBMs) are a way to design disaggregated simulation models, usually contrasted with mathematical or equation-based models, which may model a density or concentration of individuals, or otherwise lump individuals with similar attributes together in some way (Shalizi, 2006). For modeling finite numbers of individuals with significant heterogeneity in characteristics, and dynamics which could depend on such characteristics in complex ways, IBMs are a natural modeling choice whereas a representation using mathematical models (especially compartmental models) would be cumbersome or impossible (Willem et al., 2017). Even if an aggregated representation were feasible, there are many reasons why an individual-based representation is to be preferred. Synthetic data may need to produce individual level outcomes, which aggregated models by their very nature are unable to provide (Tracy et al., 2018). Other complexities, such as when events occur after a random delay whose distribution differs from a Markovian one, mean even aggregated models will need to store individual completion times, necessitating more complex simulation algorithms and data structures; in such cases it is often more straightforward to adopt an individual-based representation from the start.

`individual` is an R package which provides users a set of useful primitive elements for specifying individual based simulation models, with special attention models for infectious disease epidemiology. Users build models by specifying variables for each characteristic of individuals in the simulated population by using data structures exposed by the package. The package provides efficient methods for finding subsets of individuals based on these variables, or cohorts. Cohorts can then be targeted for variable updates or scheduled for events. Variable updates queued during a time step are executed at the end of a discrete time step, and the code places no restrictions on how individuals are allowed to interact. These data structures are designed to provide an intuitive way for users to turn their conceptual model of a system into executable code, which is fast and memory efficient.

Statement of need

In many applications, but especially epidemiology, individual-based models need to balance comprehensibility and speed. A fast model whose code is only understood by the author can be difficult to use as a basis for scientific exploration, which necessitates the development of

*Custom footnotes for e.g. denoting who the corresponding author is can be included like this.

various similar models to test different hypotheses or explore sensitivity to certain assumptions. On the other hand a clear yet slow model can be practically unusable for tasks such as uncertainty quantification or statistical inference on model parameters. `individual` provides a toolkit for epidemiologists to write models that is general enough to cover nearly all models of practical interest using simple, standardized code which is fast enough to be useful for computation heavy applications.

Based on our review of existing software below, no other library exists in the R language which provides users with a set of primitive elements for defining epidemiological models without making strong restrictions upon the type of model being simulated (e.g.; compartmental, network, etc.), or limiting users to particular mathematical forms for transition probabilities.

Design Principles

The `individual` package is written in the R language, which is a *lingua franca* in epidemiology. The package uses `Rcpp` (Eddelbuettel & François, 2011) to link to C++ source code, which underlies the data structures exposed to the user. The API for `individual` uses `R6` (Chang, 2020) classes at the R level which users call to create, update, and query variables.

Because in many epidemiological models the most important part of an individual's state can be represented as mutually exclusive classes in a finite set, `individual` uses a bitset programmed in C++ to store these data. At the R level users can call various set operations (union, intersection, complement, symmetric difference, set difference) which are implemented as bitwise operations in the C++ source. This lets users write clear, highly efficient code for updating their model, fully in R.

In contrast to other individual based modeling software, where users focus on defining a type for simulated individuals, in `individual` users instead define variables, one for each characteristic. Individual agents are only defined by their their position in each bitset which defines membership in a variable, or position in a vector of integers or floats for unbounded or floating point variables. This design is similar to a component system, a design pattern to help decouple complicated types (Nystrom, 2014). Therefore, state is represented in a disaggregated manner, and preforming operations to find and interact with cohorts of individuals benefits from fast bitwise operators. This representation of state is (to our knowledge), novel for epidemiological simulation. While Rizzi et al. (2018) proposed using a bitset to represent the state of each simulated individual, the population was still stored as types in an array.

`individual` also provides a C++ header-only interface which advanced users can link to from their R package. The header-only interface allows a user to interact with the C++ types directly, if the R interface remains too slow for their use case. This means that users can link to other R packages that expose a C/C++ interface, significantly enhancing the extensibility of `individual`'s API.

State of the field

There are currently a variety of software libraries for epidemiological simulation, both in R and other programming languages. However, we are not yet aware of a similar package written especially for epidemiology which allows simulation of generic infectious processes by providing users with a set of specialized data types and methods without making assumptions about model structure (e.g.; network, metapopulation, lattice grid, etc). Additionally, none of the reviewed software used bitset objects to optimize certain operations.

DRAFT

Non R Software

A wide variety of simulation software exist for generic agent-based models. Among the best known are Repast (North et al., 2013), Mesa (Masad & Kazil, 2015), and NetLogo (Wilensky, 1999). In the Julia programming language, Agents.jl (Vahdati, 2019) provides an efficient platform for specifying and simulation agent-based models.

Various software libraries also exist specifically for epidemiological simulation. EpiFire (Hladish et al., 2012) and SimpactCyan (Liesenborgs et al., 2019) are C++ libraries for simulation of epidemics on networks. Numerus Model Builder (Getz et al., 2018) and NOVA (Salter, 2013) use a domain specific language to define models. EMOD (Bershteyn et al., 2018) is a highly successful and complex modeling software in C++ which relies on JSON configuration files to interact with. Pathogen.jl (Angevaare et al., 2020) is a package for individual based simulation of common compartmental models.

R Software

General R Packages

Several R packages provide interfaces to other software libraries. The nlrx package provides an R interface to NetLogo (Salecker et al., 2019) to set up reproducible experiments and focuses on sensitivity analysis, and RNetLogo is a more basic interface (Thiele, 2014). NetLogoR (Bauduin et al., 2019) is a complete translation of NetLogo into R. For the Repast library, RRepast provides a sophisticated interface (García & Rodríguez-Patón, 2016). Finally, simecol (Petzoldt & Rinke, 2007) provides classes and methods to enhance reproducibility of ecological models.

Generic individual based simulation packages in R include the simmeR package for discrete event simulation (Ucar et al., 2017) which also utilizes a `R6` interface to lined C++ but is specialized for queueing type models. SpaDES (McIntire et al., 2021) is a generic discrete event simulator designed for ecological models on raster landscapes. IBMPopSim (Giorgi et al., 2020) is a sophisticated R package for simulation of continuous time individual based models, but requires uses to input C++ code as a string which is then compiled, making it difficult to interface with the existing R ecosystem.

Other packages include ibm (Oliveros-Ramos, 2016) which gives examples of individual based models in R but does not provide a generic interface, and ibmcraftr (Tun, 2016).

Epidemiological R Packages

There are a variety of packages in R designed to simulate epidemic processes on networks. SimInf (Bauer et al., 2016) is able to run very large simulations taking advantage of R's C interface to preform most computations in C, but has many constraints on the model dynamics. hybridModels (Marques et al., 2020) has similar capabilities but is fully implemented in R. epinet (Groendyke & Welch, 2018) and EpiDynamics (Oswaldo & Fernando, 2020) are other R packages with similar functionality. nosoi (Lequime et al., 2020) is an agent-based simulation framework designed to generate synthetic data for phylogenetic analysis, with specialized data structures for this purpose.

EpiILMCT (Almutiry & Deardon, 2020) and EpiILM (Warriyar et al., 2020) also simulate epidemics on networks, relying on a FORTRAN backend. SPARSEMODr (Mihaljevic et al., 2021) has similar capabilities, although with a C++ backend. These packages however restrict the transmission term (force of infection) to common mathematical forms, limiting generalizability.

DRAFT

EpiModel (Jenness et al., 2018) is perhaps the most relevant R software we have reviewed, allowing simulation of highly detailed discrete time models on networks, relying on the statnet (Krivitsky et al., 2003-2020) project for classes and algorithms. However due to its focus on directly transmitted diseases, `individual` may be more generic for other epidemiological sitautions (such as vector borne diseases). In addition it does not offer an interface for compiled code.

Overview

The primitives in the `individual` package can be separated into variables, processes, events and rendering. Each primitive is designed to simplify and optimise a common challenge in the simulation of infectious disease models.

The package is designed to allow models integrate easily with other R and C++ packages. There is no compilation of model code, and primitives are made available through `R6` (Chang, 2020) or C++ classes.

Variables

A variable represents an attribute of each individual in the model. While many variables will be dynamically updated throughout a simulation, they can also remain constant. The set of all variables is the model's state.

Variables expose methods for creating cohorts. Users can define cohorts by selecting ranges of attribute values, or combining other cohorts using efficient set operations. This simplifies much of the modelling code into performant, vectorised operations.

Because `individual` updates on a discrete time step, care must be taken when modeling processes which can cause the same variable to change state. Update conflicts in `individual` are solved by transactional updates. Each process has access to all variables and may queue updates, but variable updates are not applied until all processes have run for the current time step. This means all individuals update synchronously where conflicts (multiple updates scheduled for a single agent) are resolved by process execution order.

Processes

Processes determine the dynamics which update the model from one time step to the next. As a result of disaggregated representation of state with variable objects there is no difference between processes which depend on interactions between individuals and those which simulate an individual's internal dynamics. All processes can queue variable updates and schedule events in arbitrarily complex ways.

Users can define processes in either R or C++. Processes are implemented with closures in R, and with `std::function` in C++. This gives users flexibility to use tools in their respective ecosystems, test their processes in isolation, and trade off between development speed and performance.

`individual` provides process generators for common infectious disease dynamics. Users can parameterise these generators with their model variables to speed up development and reduce their code size. Current generators include an `infection_age_process`, a `bernoulli_process`, a `fixed_probability_multinomial_process` among others.

Because processes can schedule variable updates or events depending on the state of variables in a completely general way, waiting times for events in individual can follow any distribution. Waiting time distributions can therefore depend on time, individual level attributes of multiple

DRAFT

individuals, and the time elapsed since an event was enabled. Therefore, a very general class of non-Markovian processes can be simulated in `individual`.

Events

In some cases, users would like processes to execute at specific time steps. Events provide that functionality. Events can be scheduled before or during model simulation, they can be pre-empted, and they can be targeted to a cohort of individuals. Users can attach listener functions to these events which may queue variable updates or schedule further events when triggered. Events are useful for modelling interventions like vaccinations, or delayed biological events like incubation periods.

Like processes, listeners can be defined in R or C++. `individual` also provides listener generators like `reschedule_listener` and `update_category_listener`.

Rendering

Rendering primitives are used to record statistics from a simulation. The `Render` class combines statistics from various processes into a data frame for subsequent analysis.

Simulation loop

The simulation loop combines the primitives to run a discrete-time simulation and produce a data frame of results. It executes processes, triggered listeners and then variable updates for each time step.

The simulation loop has predictable resolutions for conflicting variable update transactions. The user specifies the order in which processes and listeners (if triggered) are executed. Variable updates are executed in FIFO order with later updates overwriting earlier ones. This results in later processes and listeners taking precedence over earlier ones. Updates produced by listeners take precedence over any updates produced from processes.

DRAFT

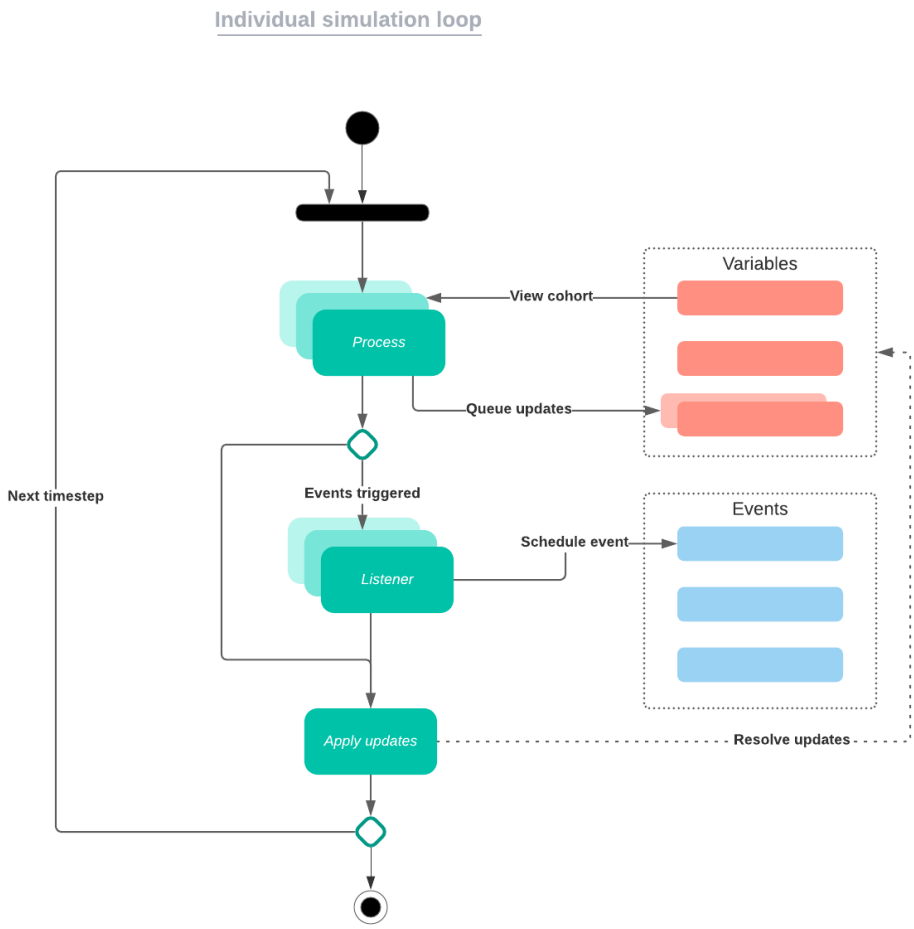

Figure 1: A flow diagram for the simulation loop

Example

To demonstrate how to use `individual` we will build and simulate a simple SIR model. For details on the mathematical construction stochastic epidemic models, please consult Allen (2017). This example follows the package vignette.

The epidemic will be simulated in a population of 1000, where 5 persons are initially infectious, whose indices are randomly sampled. The effective contact rate β will be a function of the deterministic R_0 and recovery rate γ. We also specify `dt`, which is the size of the intended time step. Because in `individual` all time steps are of unit duration, by adjusting transition rates appropriately by `dt` the unit time step can be scaled to any desired size. Because the maximum time is `tmax` the number of discrete time steps taken is `tmax/dt`.

```
N <- 1e3 # population size
I0 <- 5 # initial number of infectious individuals
S0 <- N - I0 # initial number of susceptible individuals

dt <- 0.1 # size of time step
tmax <- 100 # maximum simulation time
steps <- tmax/dt
```

```r
gamma <- 1/10 # recovery rate
R0 <- 2.5
beta <- R0 * gamma

# sample who is initially infectious
health_states <- c("S","I","R")
health_states_t0 <- rep("S",N)
health_states_t0[sample.int(n = N,size = I0)] <- "I"
```

Then the model's state is defined by a single `CategoricalVariable` object.

```r
health <- CategoricalVariable$new(
  categories = health_states,initial_values = health_states_t0
)
```

Next we define the infection process. This is a function that takes only a single argument, `t`, the current time step (unused here, but can model time-dependent processes, such as seasonality or school holiday). Within the function, we get the current number of infectious individuals, then calculate the per-capita force of infection on each susceptible person, $\lambda = \beta I/N$. Next we get a `Bitset` containing those susceptible individuals and use the sample method to randomly select those who will be infected on this time step. The probability is given by $1 - e^{-\lambda \Delta t}$. This is the same as the CDF of an exponential random variate so we use the base R function `pexp` to compute that quantity. Finally, we queue a state update for those individuals who were sampled.

```r
infection_process <- function(t){
  I <- health$get_size_of("I")
  foi <- beta * I/N
  S <- health$get_index_of("S")
  S$sample(rate = pexp(q = foi * dt))
  health$queue_update(value = "I",index = S)
}
```

Now we need to model recovery. For geometrically distributed infectious periods, we could use another process that randomly samples some individuals each time step to recover, but we'll use a `TargetedEvent` to illustrate their use. The recovery event is quite simple. The listener takes two arguments, the time step it is triggered and a `Bitset` of targeted individuals. Those individuals are scheduled for to update their health state to "R."

```r
recovery_event <- TargetedEvent$new(population_size = N)
recovery_event$add_listener(function(t, target) {
  health$queue_update("R", target)
})
```

Finally, we need to define a recovery process that queues future recovery events. We first get `Bitset` objects of those currently infectious individuals and those who have already been scheduled for a recovery. Then, using bitwise operations, we get the intersection of already infectious persons with persons who have not been scheduled, precisely those who need to have recovery times sampled and scheduled. We sample those times from `rgeom`, where the probability for recovery is $1 - e^{-\gamma \Delta t}$.

We note at this point would be possible to queue the recovery event at the same time the infection state update was made, but we separate event and process for additional illustration of how the package works.

DRAFT

```
recovery_process <- function(t){
  I <- health$get_index_of("I")
  already_scheduled <- recovery_event$get_scheduled()
  I$and(already_scheduled$not())
  rec_times <- rgeom(n = I$size(),prob = pexp(q = gamma * dt)) + 1
  recovery_event$schedule(target = I,delay = rec_times)
}
```

The last thing to do before simulating the model is rendering output to plot. We use a `Render` object which stores output from the model each time step.

```
health_render <- Render$new(timesteps = steps)
health_render_process <- categorical_count_renderer_process(
  renderer = health_render,
  variable = health,
  categories = health_states
)
```

Finally, the simulation can be run by passing objects to the `simulation_loop` function.

```
simulation_loop(
  variables = list(health),
  events = list(recovery_event),
  processes = list(infection_process,recovery_process,health_render_process),
  timesteps = steps
)
```

By default, `Render` object return `data.frame` objects which can be easily plotted and analyzed.

DRAFT

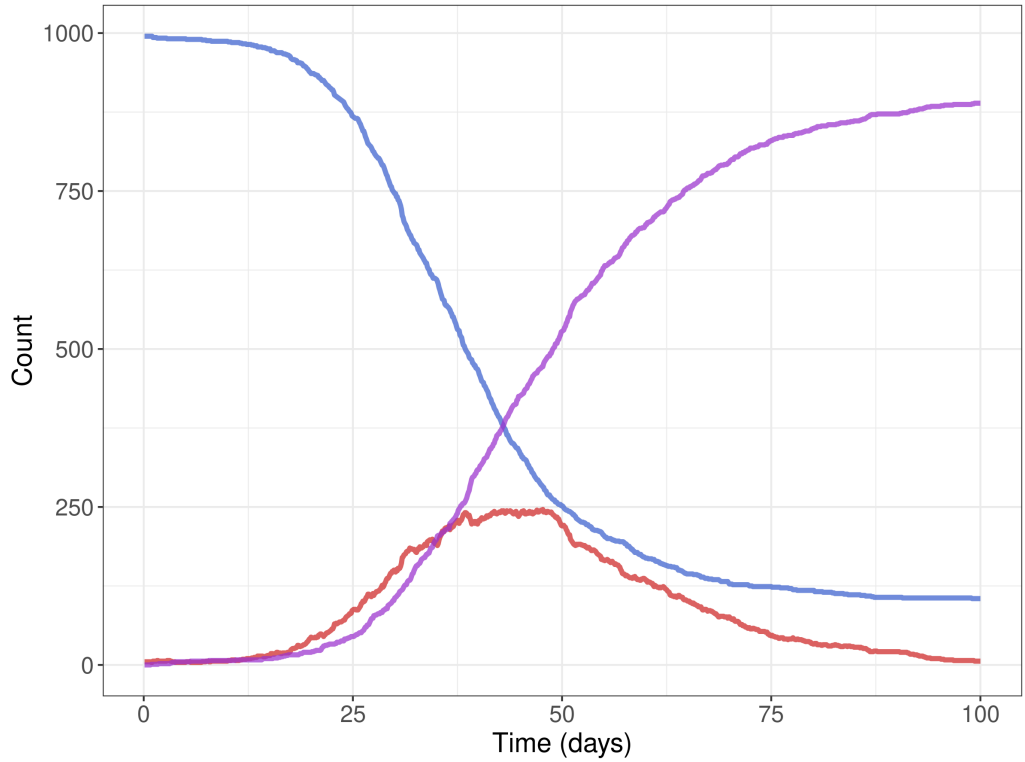

Figure 2: A simulated SIR model trajectory

Licensing and Availability

`individual` is licensed under the MIT License, with all source code stored at GitHub. Requests, suggestions, and bug reports are encouraged via filing an issue. A general guide on how to contribute to `individual` is available at the package's website.

Acknowledgements

Pete Winskill, Richard Fitzjohn, Oliver Watson

References

Allen, L. J. (2017). A primer on stochastic epidemic models: Formulation, numerical simulation, and analysis. *Infectious Disease Modelling*, *2*(2), 128–142.

Almutiry, W., & Deardon, R. (2020). *EpiILMCT: Continuous time distance-based and network-based individual level models for epidemics*. https://CRAN.R-project.org/package=EpiILMCT

Angevaare, J., Feng, Z., & Deardon, R. (2020). *Pathogen.jl: Infectious disease transmission network modelling with Julia*. https://arxiv.org/abs/2002.05850

Bauduin, S., McIntire, E. J., & Chubaty, A. M. (2019). NetLogoR: A package to build and run spatially explicit agent-based models in r. *Ecography*, *42*(11), 1841–1849.

Bauer, P., Engblom, S., & Widgren, S. (2016). Fast event-based epidemiological simulations on national scales. *The International Journal of High Performance Computing Applications*, *30*(4), 438–453.

Bershteyn, A., Gerardin, J., Bridenbecker, D., Lorton, C. W., Bloedow, J., Baker, R. S., Chabot-Couture, G., Chen, Y., Fischle, T., Frey, K., & others. (2018). Implementation and applications of EMOD, an individual-based multi-disease modeling platform. *Pathogens and Disease*, *76*(5), fty059.

Chang, W. (2020). *R6: Encapsulated classes with reference semantics*. https://CRAN.R-project.org/package=R6

Eddelbuettel, D., & François, R. (2011). Rcpp: Seamless R and C++ integration. *Journal of Statistical Software*, *40*(8), 1–18. https://doi.org/10.18637/jss.v040.i08

Ganyani, T., Faes, C., & Hens, N. (2021). Simulation and analysis methods for stochastic compartmental epidemic models. *Annual Review of Statistics and Its Application*, *8*(1), 69–88. https://doi.org/10.1146/annurev-statistics-061120-034438

García, A. P., & Rodríguez-Patón, A. (2016). Sensitivity analysis of repast computational ecology models with r/repast. *Ecology and Evolution*, *6*(24). https://doi.org/10.1002/ece3.2580

Getz, W. M., Salter, R., Muellerklein, O., Yoon, H. S., & Tallam, K. (2018). Modeling epidemics: A primer and numerus model builder implementation. *Epidemics*, *25*, 9–19.

Giorgi, D., Kaakai, S., & Lemaire, V. (2020). *IBMPopSim: Individual based model population simulation*. https://CRAN.R-project.org/package=IBMPopSim

Groendyke, C., & Welch, D. (2018). epinet: An R package to analyze epidemics spread across contact networks. *Journal of Statistical Software*, *83*(11), 1–22. https://doi.org/10.18637/jss.v083.i11

Hladish, T., Melamud, E., Barrera, L. A., Galvani, A., & Meyers, L. A. (2012). EpiFire: An open source c++ library and application for contact network epidemiology. *BMC Bioinformatics*, *13*(1), 1–12.

Jenness, S. M., Goodreau, S. M., & Morris, M. (2018). EpiModel: An r package for mathematical modeling of infectious disease over networks. *Journal of Statistical Software*, *84*.

Krivitsky, P. N., Handcock, M. S., Hunter, D. R., Butts, C. T., Klumb, C., Goodreau, S. M., & Morris, M. (2003-2020). *Statnet: Software tools for the statistical modeling of network data*. Statnet Development Team. http://statnet.org

Lequime, S., Bastide, P., Dellicour, S., Lemey, P., & Baele, G. (2020). Nosoi: A stochastic agent-based transmission chain simulation framework in r. *Methods in Ecology and Evolution*, *11*(8), 1002–1007.

Liesenborgs, J., Hendrickx, D. M., Kuylen, E., Niyukuri, D., Hens, N., & Delva, W. (2019). SimpactCyan 1.0: An open-source simulator for individual-based models in HIV epidemiology with r and python interfaces. *Scientific Reports*, *9*(1), 1–13.

Marques, F. S., Grisi-Filho, J. H. H., Silva, J. C. R., Almeida, E. C., & Júnior, J. L. S. (2020). hybridModels: An R package for the stochastic simulation of disease spreading in dynamic networks. *Journal of Statistical Software*, *94*(6), 1–32. https://doi.org/10.18637/jss.v094.i06

Masad, D., & Kazil, J. (2015). MESA: An agent-based modeling framework. *14th PYTHON in Science Conference*, 53–60.

McIntire, E., Chubaty, A., Luo, Y., Bauduin, S., Cumming, S., & Marchal, J. (2021). *PredictiveEcology/SpaDES: v2.0.6* (Version v2.0.6) [Computer software]. Zenodo. https://doi.org/10.5281/zenodo.4441210

DRAFT

Mihaljevic, J. R., Borkovec, S., Ratnavale, S., Hocking, T. D., Banister, K. E., Eppinger, J. E., Hepp, C. M., & Doerry, E. (2021). SPARSEMODr: Rapid simulations of spatially explicit and stochastic models of infectious disease. *medRxiv*. https://doi.org/10.1101/2021.05.13.21256216

North, M. J., Collier, N. T., Ozik, J., Tatara, E. R., Macal, C. M., Bragen, M., & Sydelko, P. (2013). Complex adaptive systems modeling with repast simphony. *Complex Adaptive Systems Modeling*, *1*(1), 1–26.

Nystrom, R. (2014). *Game programming patterns*. Genever Benning.

Oliveros-Ramos, R. (2016). *Ibm: Individual based models in r*. https://CRAN.R-project.org/package=ibm

Oswaldo, S. B., & Fernando, S. M. (2020). *EpiDynamics: Dynamic models in epidemiology*. https://CRAN.R-project.org/package=EpiDynamics

Petzoldt, T., & Rinke, K. (2007). Simecol: An object-oriented framework for ecological modeling in r. *Journal of Statistical Software*, *22*(9), 1–31.

Rizzi, R. L., Kaizer, W. L., Rizzi, C. B., Galante, G., & Coelho, F. C. (2018). Modeling direct transmission diseases using parallel bitstring agent-based models. *IEEE Transactions on Computational Social Systems*, *5*(4), 1109–1120.

Salecker, J., Sciaini, M., Meyer, K. M., & Wiegand, K. (2019). The nlrx r package: A next-generation framework for reproducible NetLogo model analyses. *Methods in Ecololgy and Evolution*, *00*, 2041–210X. https://doi.org/10.1111/2041-210X.13286

Salter, R. M. (2013). Nova: A modern platform for system dynamics, spatial, and agent-based modeling. *Procedia Computer Science*, *18*, 1784–1793.

Shalizi, C. R. (2006). Methods and techniques of complex systems science: An overview. *Complex Systems Science in Biomedicine*, 33–114.

Thiele, J. C. (2014). R marries NetLogo: Introduction to the RNetLogo package. *Journal of Statistical Software*, *58*(2), 1–41.

Tracy, M., Cerdá, M., & Keyes, K. M. (2018). Agent-based modeling in public health: Current applications and future directions. *Annual Review of Public Health*, *39*(1), 77–94. https://doi.org/10.1146/annurev-publhealth-040617-014317

Tun, S. T. T. (2016). *Ibmcraftr: Toolkits to develop individual-based models in infectious disease*. https://CRAN.R-project.org/package=ibmcraftr

Ucar, I., Smeets, B., & Azcorra, A. (2017). Simmer: Discrete-event simulation for r. *arXiv Preprint arXiv:1705.09746*.

Vahdati, A. R. (2019). Agents. Jl: Agent-based modeling framework in julia. *Journal of Open Source Software*, *4*(42), 1611.

Warriyar, V. K. V., Almutiry, W., & Deardon, R. (2020). *EpiILM: Spatial and network based individual level models for epidemics*. https://CRAN.R-project.org/package=EpiILM

Wilensky, U. (1999). *NetLogo* [Http://ccl.northwestern.edu/netlogo/]. Center for Connected Learning; Computer-Based Modeling. http://ccl.northwestern.edu/netlogo/

Willem, L., Verelst, F., Bilcke, J., Hens, N., & Beutels, P. (2017). Lessons from a decade of individual-based models for infectious disease transmission: A systematic review (2006-2015). *BMC Infectious Diseases*, *17*(1), 1–16. https://doi.org/10.1186/s12879-017-2699-8

DRAFT